

Transfer Attacks in Face Recognition

ATT: Adaptive Token Tuning

Keshav Raj
IIIT Delhi
18 June 2026



Reason for ATT?

Why we chose Adaptive Token Tuning

- Not present in intern baseline
- Referred in the intern assignment
- To understand ML architecture
- Easier to implement than other attacks
- To understand tuning and embedding

code

Github Repository

Explore the implementation details and source code.



Understanding ATT

1. Adaptive Targeting

Actively moves source face embedding toward target while preserving perturbation constraints.

2. Embedding Alignment

Directly optimizes similarity between Adversarial and Target image embeddings.

3. Momentum

Ensures the attack follows a more stable direction for better convergence.

4. Impersonation Attack

Moves the source embedding specifically towards the target identity's embedding.

5. Dodging Attack

Pushes the embedding away from the genuine identity to avoid recognition.

Adapting ATT to Face Verification

1. Objective Shift

Replaced Classification Objective with Face embedding objective

2. Optimization Target

Used cosine Similarity as the Optimisation target

3. Verification Logic

Added Verification Thresholds

4. Attack Profiles

Added impersonation attack and Dodging attack

5. Transferability Analysis

Understanding transferability across other Victim models

Code Integration of ATT

ALL_ATTACKS

Registered "att" in the global attack dictionary for management.

ATTACK_COLS

Added the "att_path" to the attack columns

```
'PGD',  
'MI_FGSM',  
'TI_FGSM',  
'SI_NI_FGSM',  
'MI_ADMIX_DI_TI',  
'ATT',  
  
]  
  
ATTACK_COLS = {  
    'PGD': 'pgd_path',  
    'MI_FGSM': 'mi_fgsm_path',  
    'TI_FGSM': 'ti_fgsm_path',  
    'SI_NI_FGSM': 'si_ni_fgsm_path',  
    'MI_ADMIX_DI_TI': 'mi_admix_di_ti_path',  
    'ATT': 'att_path',  
}
```

Execution Flow: ATT Attack

01 Start with original image

02 Initialise momentum

03 Compute step size

04 Normalize target embedding

05 Start ATT iterations

06 Track gradients

07 Watch image

08 Extract embedding

09 Compute cosine similarity

10 Build attack loss

11 Compute gradient

12 ATT Gradient Variance Reduction

13 ATT Feature Preservation

14 MI-FGSM Gradient Normalization

15 Momentum Update

16 Update adversarial image

17 Enforce epsilon constraint

18 Keeping image valid

```
def att_attack(model,x,tgt_emb,attack_type):
    adv=tf.identity(x)

    g=tf.zeros_like(x)

    alpha=EPSILON/NUM_ITER

    tgt_emb=tf.nn.l2_normalize(tgt_emb, axis=1)

    for _ in range(NUM_ITER):

        with tf.GradientTape() as tape:
            tape.watch(adv)
            emb=compute_embedding(model, adv)
            cos=tf.reduce_sum(emb * tgt_emb, axis=1)
            loss=attack_loss(cos, attack_type)

        grad=tape.gradient(loss, adv)
        #Gradient Variance Reduction of ATT
        grad_var=tf.math.reduce_variance(grad)
        grad=grad/(tf.sqrt(grad_var)+1e-8)
        # ATT: Feature Preservation
        mean_grad=tf.reduce_mean(tf.abs(grad))
        strong_mask=tf.abs(grad)>mean_grad
        grad = tf.where(strong_mask,grad * 0.7,grad)
        # MI-FGSM normalization
        grad=grad/(tf.reduce_mean(tf.abs(grad)) + 1e-8)
        # Momentum
        g=DECAV*g+grad
        # Update image
        adv=adv+alpha*tf.sign(g)
        adv=tf.clip_by_value(adv,x-EPSILON,x+EPSILON)
        adv=tf.clip_by_value(adv,-1.0,1.0)
```

Final Result & Output Analysis

Attack Method	Breach Rate
SI-NI-FGSM	31.46%
MI-FGSM	26.04%
ATT	24.46%
MI-ADMIX-DI-TI	23.96%
TI-FGSM	20.00%
PGD	16.46%

Key Findings

Competitive Advantage: ATT successfully outperformed several established methods including MI-ADMIX-DI-TI, PGD, and TI-FGSM.

Current Limitations: While highly competitive with MI-FGSM and SI-NI-FGSM, it could not yet surpass their breach rates in this evaluation.

What I learned

- Understood pytorch and Numpy for the assignment.
- Face verification and analysis repository.
- Reverse engineering the evaluation scheme.
- Facial embeddings and cosine similarity

THANK YOU

Keshav Raj
IIIT Delhi
18th June 2026